Cross-Border Tax Intelligence with Graph-RAG

Technical Architecture, Knowledge Modeling, Retrieval Workflows, and Reference Implementation

Abstract

This reference architecture describes a tax intelligence system combining conventional Retrieval-Augmented Generation with graph-based retrieval. Jurisdiction-local questions can use scoped semantic retrieval. Cross-border questions require relationship-aware retrieval across tax residency, asset location, source-country taxation, residence-country taxation, treaties, withholding, foreign tax credits, refunds, reporting obligations, legal authority, and effective dates.

Contents

1 Problem Definition

A local query such as "How are dividends from Indian listed companies taxed for an Indian resident?" may primarily require retrieval from one jurisdictional corpus:

$$\text{Query} \rightarrow \text{India Tax VectorDB} \rightarrow \text{Legal Evidence} \rightarrow \text{LLM}.$$

A cross-border query such as "An Indian tax resident owns US shares, receives dividends with US tax withheld, and wants to understand Indian treatment and possible relief" requires connected knowledge across multiple legal systems.

The architectural question therefore changes from:

> Which documents are semantically similar to the question?

to:

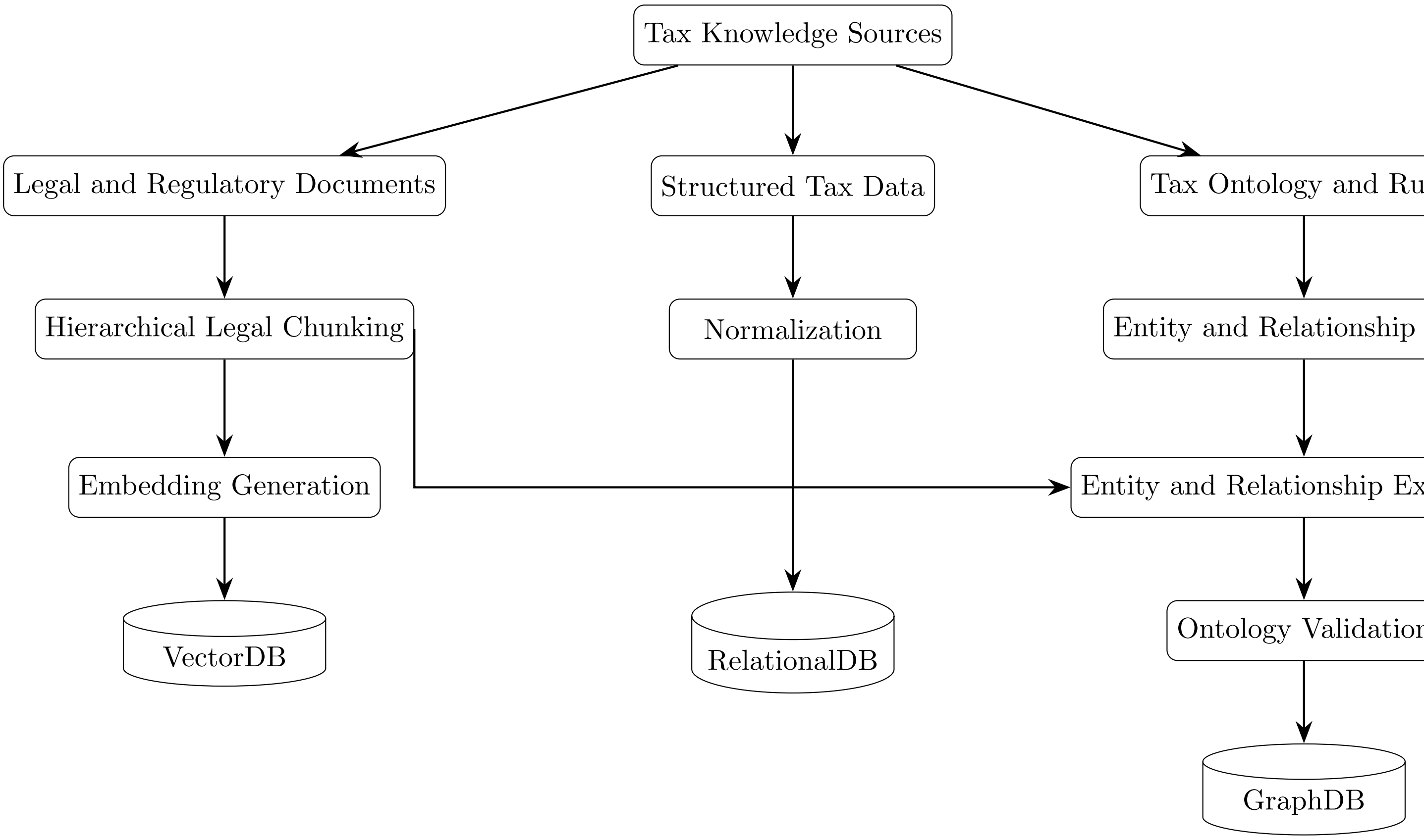
> Which jurisdictions, legal concepts, treaties, tax events, and reporting obligations are structurally connected to this fact pattern and must therefore be considered together?

2 Storage Architecture

Storage	Primary responsibility
Vector Database	Semantic retrieval of statutes, regulations, tax authority guidance, treaty text, procedures, and explanatory material.
Graph Database	Entities and relationships among jurisdictions, assets, income, tax events, treaties, credits, refunds, and reporting obligations.
Relational Database	Exact rates, thresholds, dates, filing parameters, canonical identifiers, normalized rule attributes, and deterministic values.
Document/Object Store	Original source documents, PDFs, historical versions, and raw ingestion material.

> The VectorDB retrieves semantically relevant evidence. The GraphDB identifies structurally connected legal domains. The RelationalDB supplies exact structured parameters and time-dependent values.

3 High-Level Architecture



4 Knowledge Domains and Vector Namespaces

A single physical VectorDB may use namespaces, collections, or metadata partitions:

```
tax-law/
+-- india/
+-- united-states/
+-- germany/
+-- united-kingdom/
+-- singapore/
+-- uae/
+-- other-jurisdictions/
+-- treaties/
+-- international-tax/
+-- reporting/
```

The GraphDB determines which namespaces are relevant to a cross-border fact pattern.

5 Graph Ontology

The initial ontology should remain deliberately small. Its purpose is retrieval orchestration, not an attempt to model all global tax law.

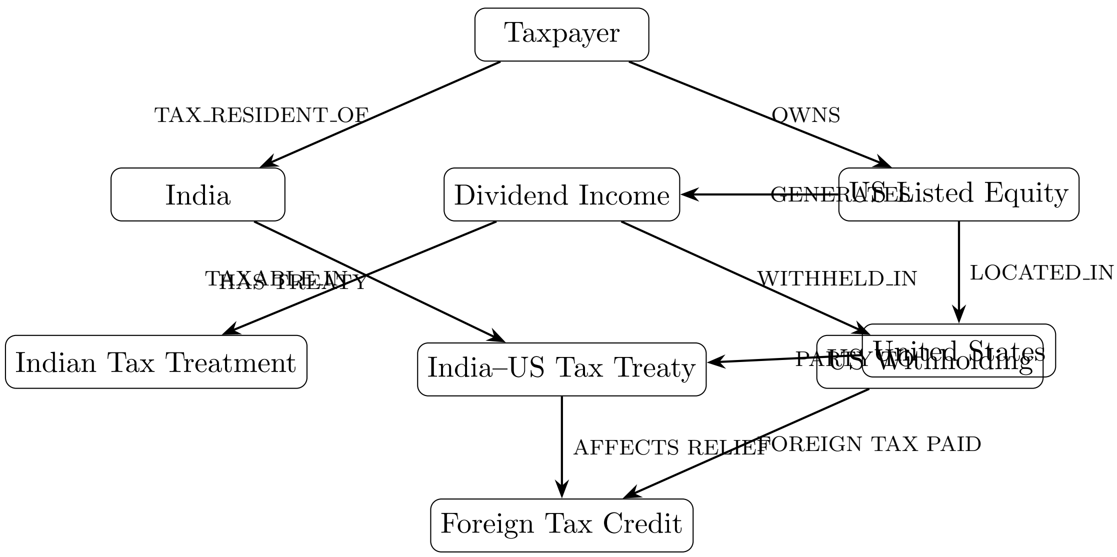
5.1 Primary Entities

Entity	Examples
Taxpayer	Individual, company, trust, partnership
Tax Residence	Resident, non-resident, jurisdiction-specific status
Jurisdiction	India, United States, Germany, Singapore
Asset	Equity, real estate, bond, bank account, retirement account
Income	Dividend, interest, capital gain, rental income, royalty
Tax Event	Sale, distribution, vesting, exercise, redemption
Tax Rule	Statute, regulation, administrative rule
Tax Treaty	Bilateral or multilateral agreement
Treaty Article	Dividend, capital gain, or relief article
Tax Credit	Foreign tax credit or related relief
Refund Mechanism	Withholding refund or recovery procedure
Reporting Requirement	Foreign asset or tax-return disclosure

5.2 Core Relationships

```
Taxpayer --TAX_RESIDENT_OF--> Jurisdiction
Taxpayer --OWNS--> Asset
Asset --LOCATED_IN--> Jurisdiction
Asset --GENERATES--> Income
Income --TAXABLE_IN--> Jurisdiction
Income --SOURCE_TAXABLE_IN--> Jurisdiction
Tax --WITHHELD_BY--> Jurisdiction
Jurisdiction --HAS_TREATY_WITH--> Jurisdiction
Treaty --CONTAINS--> Treaty Article
Foreign Tax Paid --MAY_QUALIFY_FOR--> Foreign Tax Credit
Taxpayer --SUBJECT_TO--> Reporting Requirement
Tax Rule --APPLIES_TO--> Income
```

6 Reference Graph: Indian Resident with US Equity

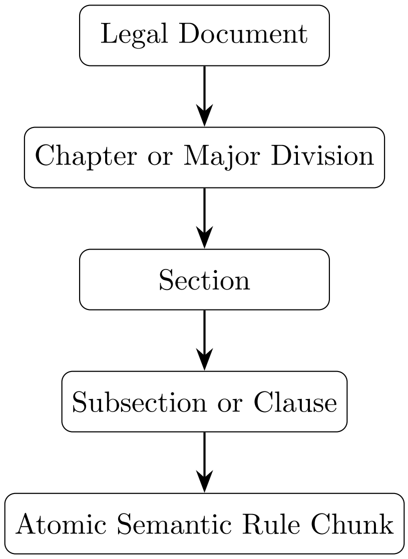


The graph does not itself determine final tax liability. It identifies the legal neighborhoods that should participate in evidence retrieval.

7 Chunking Strategy

Fixed-size chunking alone is insufficient because legal provisions depend on hierarchy, provisos, exceptions, definitions, cross-references, and effective dates.

The recommended structure is hierarchical:



Example metadata:

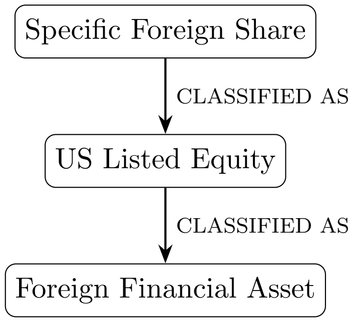
```
chunk_id: IN_CAP_GAIN_001
jurisdiction: India
tax_domain: Capital Gains
income_type: Capital Gain
asset_type: Listed Equity
taxpayer_type: Individual
legal_source: Income Tax Act
section: ...
effective_from: YYYY-MM-DD
effective_to: null
authority_level: Primary
entity_ids:
  - jurisdiction_india
  - income_capital_gain
  - asset_listed_equity
```

8 Enrichment Strategy

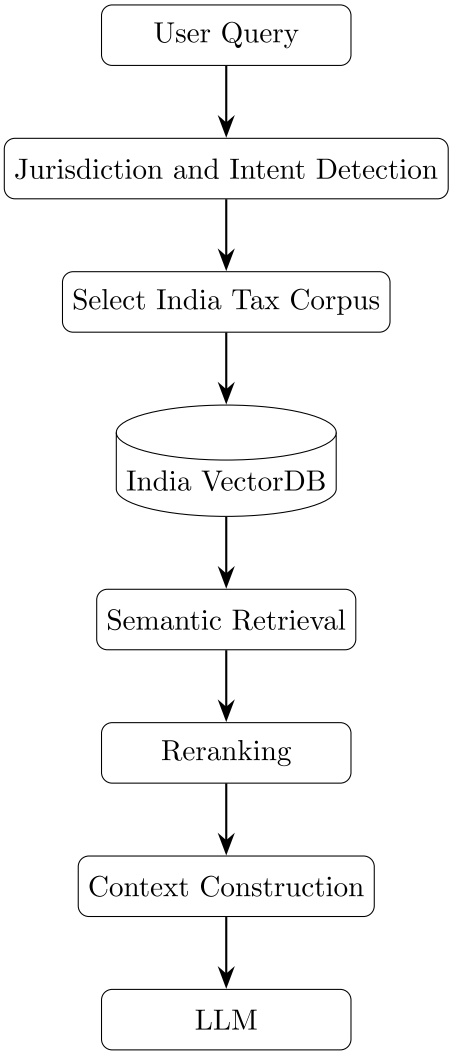
Each chunk should be enriched with:

- jurisdiction and tax authority;
- legal hierarchy;
- tax domain, income type, and asset type;
- taxpayer category;
- effective period;
- authority level;
- canonical entity identifiers;
- source document and section provenance.

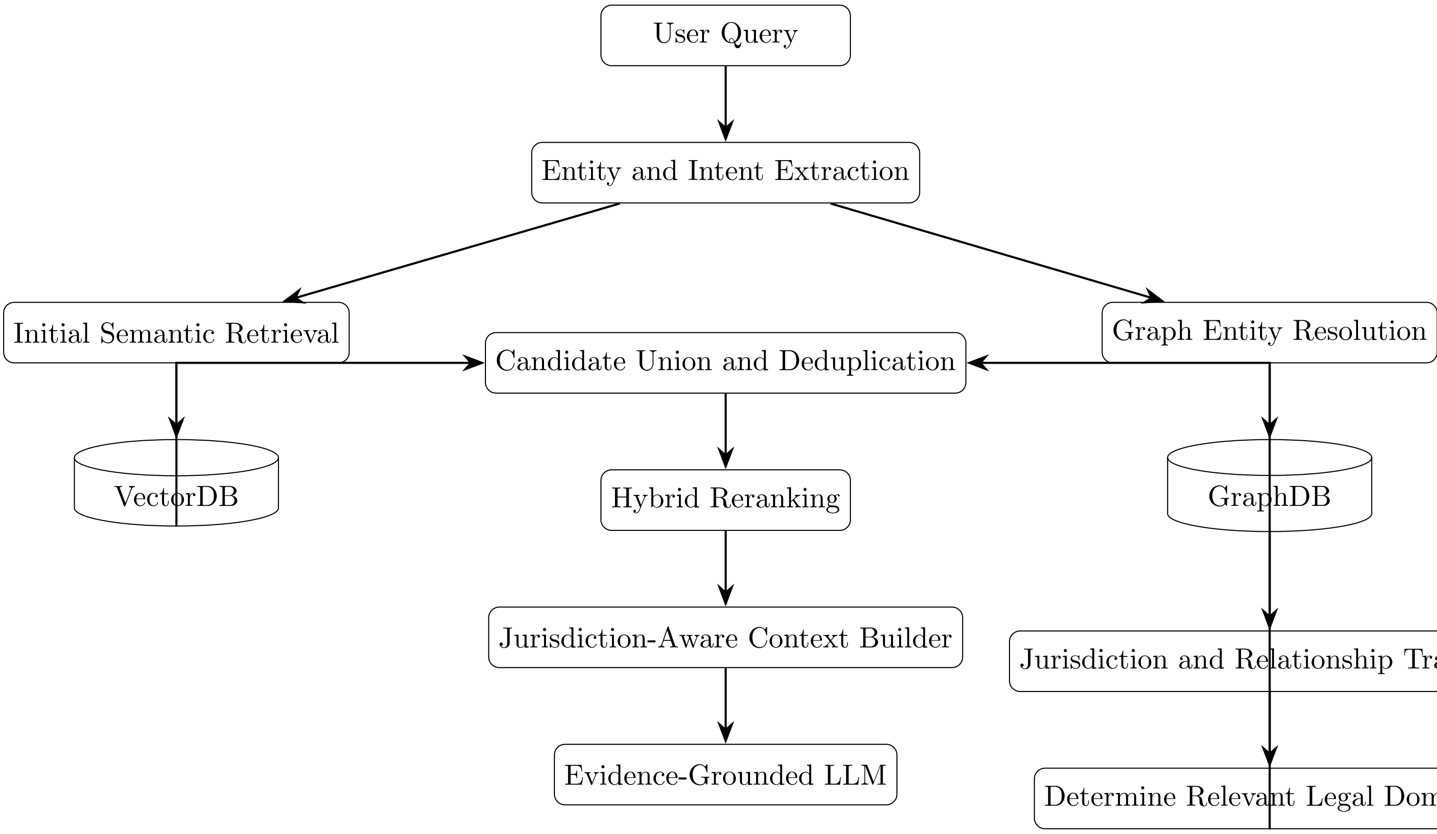
Asset normalization can map user language to legal categories:



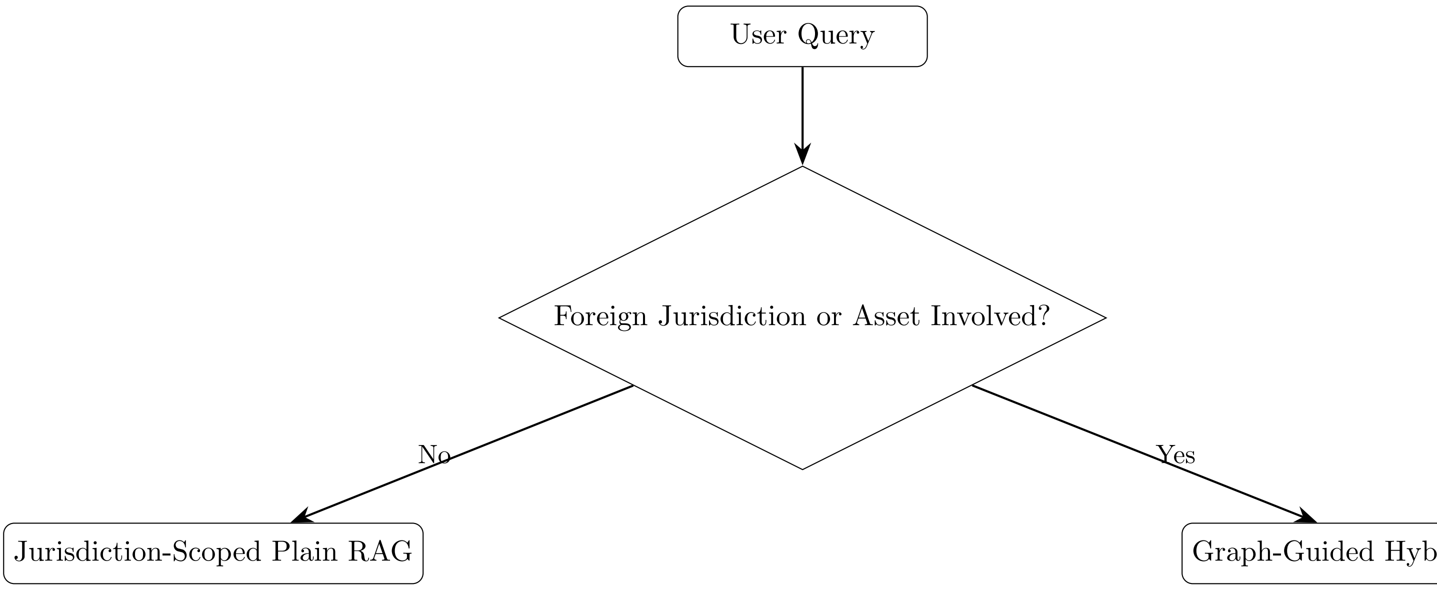
9 Local Plain RAG Workflow



10 Cross-Border Graph-RAG Workflow



11 Query Routing Logic



Suggested classes:

```
LOCAL
CROSS_BORDER
MULTI_JURISDICTION
TREATY_RELATED
FOREIGN_TAX_CREDIT
FOREIGN_ASSET_REPORTING
FOREIGN_REFUND
```

12 Graph-Guided Jurisdiction Expansion

For an Indian resident holding a foreign asset, traversal can identify:

1. residence jurisdiction;
2. asset or source jurisdiction;
3. relevant income type;
4. connected treaty;
5. foreign withholding or source taxation;
6. domestic treatment;
7. foreign tax credit or refund mechanisms;
8. reporting obligations.

The retrieval plan may therefore become:

```
Search India Tax Corpus
+
Search Foreign Jurisdiction Tax Corpus
+
Search Bilateral Treaty Corpus
+
Search Foreign Tax Credit Corpus
+
Search Reporting / Documentation Corpus
```

This is the central Graph-RAG advantage: the graph makes the required knowledge expansion explicit instead of hoping every necessary jurisdictional document ranks highly in a global semantic search.

13 Candidate Generation and Fusion

Let:

$$C = C_{\text{vector}} \cup C_{\text{graph}} \cup C_{\text{structured}}.$$

Here:

- C_{vector} contains semantic candidates;
- C_{graph} contains evidence linked to relevant entities and graph paths;
- $C_{\text{structured}}$ contains exact rules, dates, thresholds, or other deterministic records.

Every candidate should preserve provenance:

```
chunk_id
source_corpus
jurisdiction
entity_ids
graph_path
vector_score
effective_from
effective_to
authority_level
source_document_id
section_reference
```

14 Hybrid Ranking

A conceptual score is:

$$Score(c) = \alpha S_v(c) + \beta S_g(c) + \gamma S_j(c) + \delta S_t(c) + \epsilon S_a(c)$$

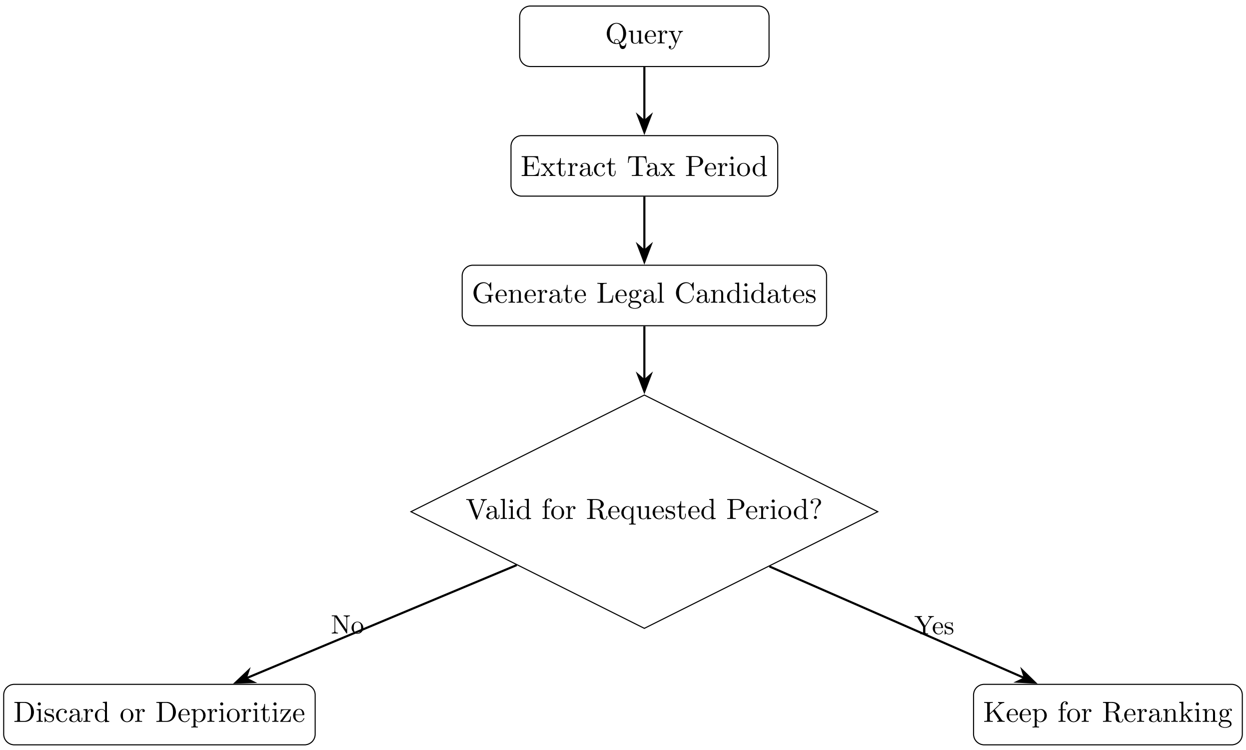
where:

- S_v is semantic relevance;
- S_g is graph or path relevance;
- S_j is jurisdiction relevance;
- S_t is temporal validity;
- S_a is source authority.

In practice, candidate generation followed by a dedicated reranker may be easier to maintain than a heavily hand-tuned formula.

15 Temporal Validity and Authority

Tax law is time-dependent. Candidate retrieval must identify the relevant tax period and filter or deprioritize rules outside that period.



A conceptual authority hierarchy is:

Primary Statute/Treaty > Regulation > Tax Authority Guidance > Official Procedure > Secondary Commentary

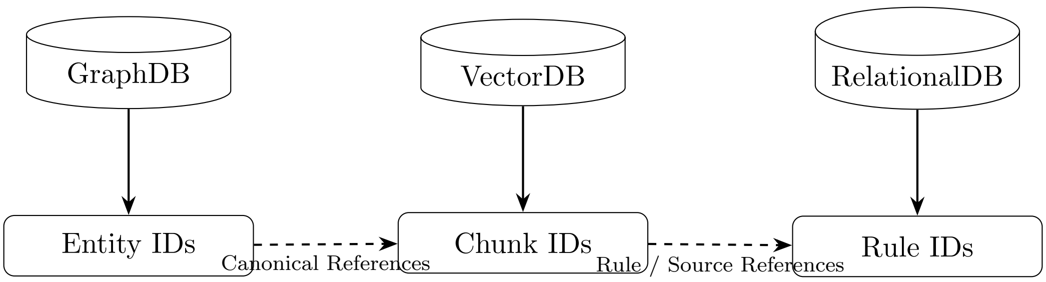
16 Structured Context Construction

The final prompt context should be organized by legal role rather than as one mixed list of chunks:

```
QUESTION

FACT PATTERN
Taxpayer Residence: ...
Asset Jurisdiction: ...
Asset Type: ...
Income: ...
Foreign Tax Event: ...

RELEVANT GRAPH PATH

SOURCE-JURISDICTION EVIDENCE

RESIDENCE-JURISDICTION EVIDENCE

TREATY EVIDENCE

FOREIGN TAX CREDIT / RELIEF EVIDENCE

REFUND EVIDENCE

REPORTING REQUIREMENTS

MISSING FACTS AND CONDITIONS
```

17 Cross-Store Linkage

All stores should use stable canonical identifiers:



This supports both:

$$\text{Graph Entity} \rightarrow \text{Supporting Chunks}$$

and:

$$\text{Chunk} \rightarrow \text{Entities} \rightarrow \text{Graph Neighborhood.}$$

18 Plain RAG versus Graph-RAG

Capability	Plain RAG	Graph-RAG
Primary signal	Semantic similarity	Semantic similarity plus relationship structure
Jurisdiction selection	Query-driven	Graph-guided expansion
Cross-border links	Implicit in text	Explicitly modeled
Treaty discovery	Similarity-dependent	Traversable jurisdiction relationships

Capability	Plain RAG	Graph-RAG
Asset classification	Metadata or inference	Graph taxonomy
Multi-hop analysis	Depends on retrieving all passages	Paths identify connected domains
Candidate expansion	Top-K similarity	Vector plus graph-derived candidates
Context	Ranked chunks	Jurisdiction and path-aware evidence
Explainability	Passage provenance	Passage plus relationship-path provenance

19 Evaluation

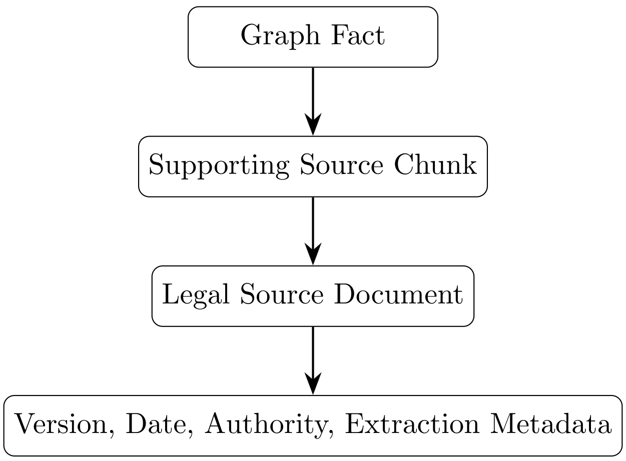
Evaluation should compare local and cross-border query classes:

- local factual retrieval;
- cross-border taxation;
- treaty-related retrieval;
- foreign tax credit;
- foreign asset reporting;
- refund analysis;
- multi-hop dependency;
- historical tax-period queries.

Metrics should include Context Precision, Context Recall, Entity Resolution Accuracy, Relationship Precision and Recall, Graph Path Accuracy, Jurisdiction Coverage, Temporal Validity Accuracy, Source Authority Compliance, Answer Faithfulness, Answer Correctness, Latency, and Token Cost.

20 Governance and Provenance

Every material graph fact should ideally be traceable:



Recommended provenance:

```
source_document_id
source_chunk_id
jurisdiction
legal_reference
effective_from / effective_to
authority_level
ontology_version
rulebook_version
extractor_version
confidence
ingestion_timestamp
```

21 Phased Implementation Plan

21.1 Phase 1: India-Only Plain RAG

Implement source ingestion, hierarchical chunking, metadata extraction, embeddings, VectorDB retrieval, reranking, evidence-grounded answers, and baseline evaluation.

21.2 Phase 2: Add One Foreign Jurisdiction

Add a controlled cross-border domain, for example:

```
India + United States + India--US Treaty Corpus
```

21.3 Phase 3: Add the Graph

Start with:

```
Taxpayer, Jurisdiction, Asset, Income,
Tax Event, Tax Rule, Treaty,
Tax Credit, Reporting Requirement
```

21.4 Phase 4: Add Cross-Store Linkage

Connect graph nodes, vector metadata, structured records, and source documents through canonical IDs.

21.5 Phase 5: Add Query Routing

```
LOCAL -> scoped Vector RAG
CROSS_BORDER -> Graph-guided hybrid retrieval
TREATY_RELATED -> graph traversal + treaty retrieval
FOREIGN_TAX_CREDIT -> domestic + foreign + treaty retrieval
FOREIGN_ASSET_REPORTING -> asset graph + reporting retrieval
```

21.6 Phase 6: Parallel Hybrid Retrieval

Implement semantic retrieval, entity resolution, graph traversal, jurisdiction expansion, structured retrieval, candidate union, and deduplication.

21.7 Phase 7: Ranking and Context

Add temporal filtering, authority weighting, graph relevance, reranking, jurisdiction-aware context construction, provenance, and missing-fact detection.

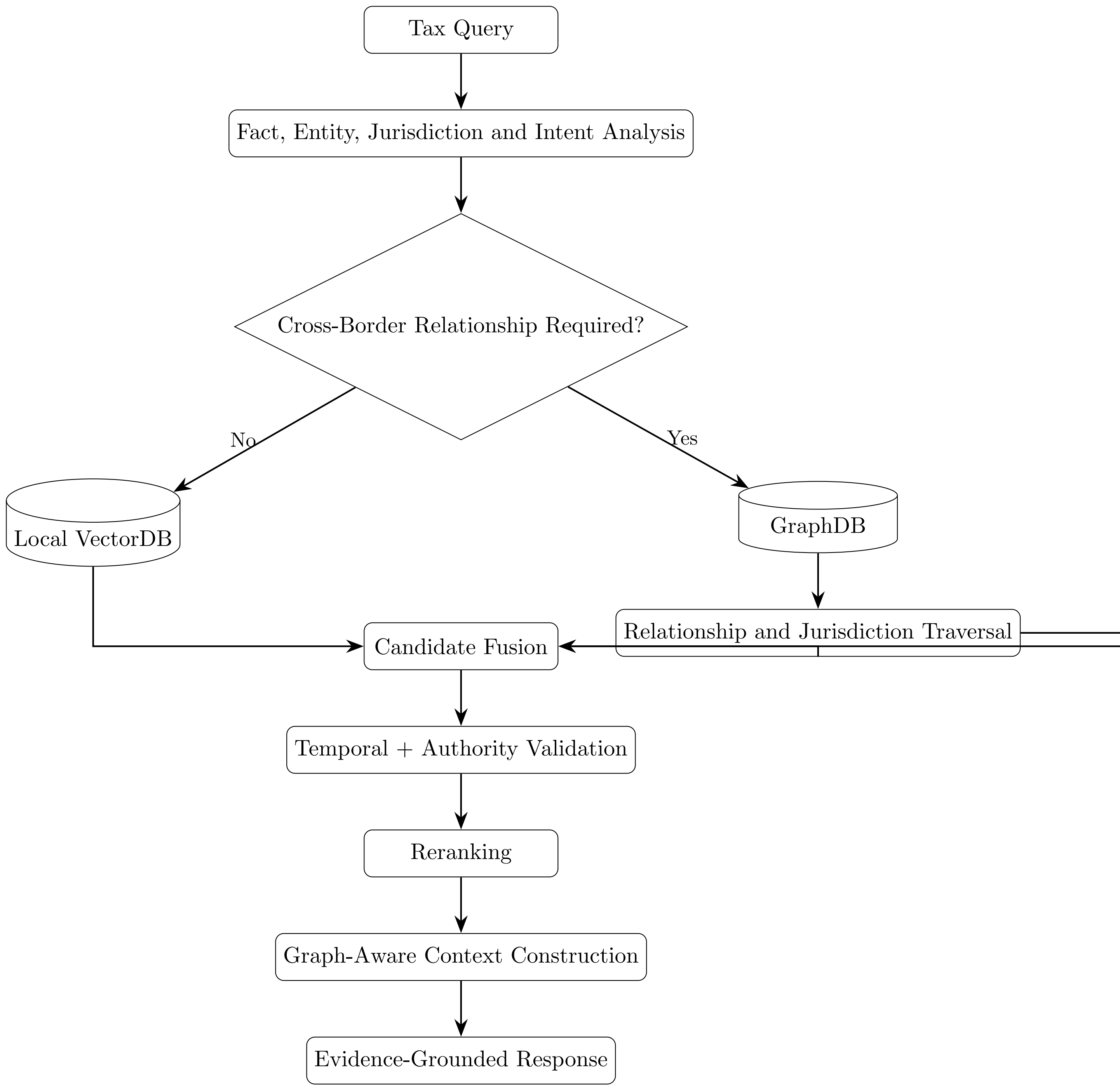
21.8 Phase 8: Regression Evaluation

Version and evaluate changes to ontology, extraction rules, embedding models, retrieval logic, traversal logic, rerankers, and context construction.

22 Reference Repository Structure

```
cross-border-tax-rag/
+-- schemas/
| +-- tax_ontology.yaml
| +-- graph_schema.yaml
+-- rules/
| +-- entity_rules.yaml
| +-- relationship_rules.yaml
+-- sources/
| +-- india/
| +-- united_states/
| +-- treaties/
+-- ingestion/
| +-- legal_parser.py
| +-- chunker.py
| +-- metadata_extractor.py
| +-- entity_extractor.py
| +-- relationship_extractor.py
| +-- validator.py
+-- storage/
| +-- vector_store.py
| +-- graph_store.py
| +-- relational_store.py
+-- retrieval/
| +-- query_classifier.py
| +-- entity_resolver.py
| +-- vector_retriever.py
| +-- graph_retriever.py
| +-- structured_retriever.py
| +-- candidate_fusion.py
| +-- temporal_filter.py
| +-- authority_filter.py
| +-- reranker.py
+-- context/
| +-- context_builder.py
| +-- provenance.py
+-- evaluation/
| +-- datasets/
| +-- retrieval_eval.py
| +-- graph_eval.py
| +-- regression_tests/
```

23 Final Reference Architecture



24 Summary

A jurisdiction-local tax question can often use scoped semantic retrieval. A cross-border question requires the retrieval system to identify and connect residence jurisdiction, asset location, source-country treatment, residence-country treatment, treaty relationships, foreign tax paid, possible relief, refund paths, reporting obligations, effective dates, and source authority.

The GraphDB supplies the structural model used to determine the retrieval neighborhood. The VectorDB retrieves source-backed legal evidence from that neighborhood. The RelationalDB provides exact structured values and temporal parameters. The retrieval orchestration layer fuses these signals, validates legal applicability, reranks evidence, and builds a structured context for the language model.